

ASSETRA

Project Report and Technical Documentation

A Django-Based Estate and Asset Management System for Law Firms

Prepared from the implemented project source code

Date: 02 May 2026

Abstract

Assetra is a full-stack Django application developed to support law firms in managing clients, estate assets, beneficiaries, uploaded documents, user roles, approvals, and audit records within a firm-scoped workspace. The project combines a server-rendered user interface with a REST API built using Django REST Framework. The system focuses on structured estate administration, approval workflows for asset distributions, secure access control, and traceable internal operations

Table of Contents (Overview)

1. Module 1 - Problem Definition and Planning
2. Module 2 - Frontend Development
3. Module 3 - Backend and Database Integration
4. Module 4 - Integration and Deployment Documentation
5. Project Report Sections: architecture, schema, security, testing, deployment, limitations, conclusion, and references

Module 1 - Problem Definition and Planning

1. Problem Statement

Small and medium law firms often manage estate information across spreadsheets, email threads, paper notes, and disconnected document folders. This creates operational issues such as duplicated data entry, weak traceability, difficult collaboration among staff, and limited control over who can approve or modify sensitive estate records. Assetra was designed to solve this by centralizing client, asset, beneficiary, document, approval, and audit operations in one application.

2. Objectives

- Create a centralized legal-tech platform for estate and asset administration.
- Support firm-scoped multi-user access with role-based restrictions.
- Track clients, beneficiaries, assets, and asset distributions in a structured way.
- Provide a document repository linked to clients and assets.
- Implement an approval workflow for distribution decisions.
- Maintain an audit trail for important business actions.
- Expose the backend through a documented REST API in addition to the web UI.

3. Stakeholder Analysis

- Firm Administrators: manage users, oversee operations, access all modules, and handle sensitive updates such as closed-estate administration.
- Lawyers: manage estate records, create and update legal data, and in senior roles approve distributions.

- Interns: access non-administrative operational data with more limited control over destructive actions.
- Clients of the law firm: not direct users of the system, but the core business entities around which estate records are maintained.
- System Maintainers/Developers: responsible for extending, securing, testing, and deploying the application.

4. System Scope

The current project scope includes authentication, dashboard summaries, client management, beneficiary management, asset management, document management, asset distribution approvals, user management, audit logs, and REST API access. Out of scope at this stage are payment systems, client self-service portals, digital signatures, advanced reporting, notification engines, and production-grade cloud automation.

5. Planning Summary

The project is structured around four implementation modules. Planning prioritized a strong domain model first, followed by web interface construction, backend API integration, and finally repository/deployment documentation. A layered approach was used so that models define the business rules, serializers/forms validate inputs, views coordinate workflows, and templates present the user interface.

Module 2 - Frontend Development

1. Frontend Approach

The frontend is implemented using Django templates, shared base layouts, CSS styling, and Django class-based web views. Instead of using a JavaScript SPA, the project follows a server-rendered architecture. This keeps the stack simpler while still providing a professional internal dashboard experience for firm staff.

2. UI Structure

- Shared layouts: `base.html` and `base_minimal.html` organize the application shell and simpler pages.
- Authentication pages: login and signup pages for workspace access.
- Dashboard page: summary cards, quick actions, and recent activity feed.
- Operational pages: clients, client detail, documents, approvals, users, and audit log.
- Form pages: add/edit client, asset, and beneficiary workflows.

3. Frontend Design Characteristics

- Firm-style internal dashboard layout with sidebar navigation.
- Reusable card, table, badge, and panel components in a central stylesheet.
- Responsive layout handling for smaller screens.

- Action-focused navigation so common operations are easy to reach from the dashboard.
- Protected document links routed through the backend instead of direct media URL exposure.

4. Frontend Modules and Responsibilities

Frontend Module	Responsibility
Authentication UI	Login and signup workflows using Django auth views and forms.
Dashboard	Displays aggregate statistics, quick actions, and recent audit activity.
Client Workspace	Shows client profile, assets, beneficiaries, distributions, and linked documents.
Documents Page	Lists documents, allows uploads, and routes file access through a protected endpoint.
Approvals Page	Provides a review queue and summary panels for pending distribution actions.
Users Page	Allows administrators to manage internal user accounts and roles.
Audit Log Page	Displays operational records for internal accountability and review.

Module 3 - Backend and Database Integration

1. Backend Overview

The backend is implemented with Django and Django REST Framework. The project separates API-oriented logic from web-page logic. API endpoints are defined in `core/views.py`, while server-rendered page workflows live in `core/web_views.py`. Validation is handled through both Django forms and DRF serializers, and role-based access control is enforced in a dedicated permissions module.

2. Backend Layers

- Models: define entities, relationships, and core business rules.
- Forms: validate and save server-rendered form inputs.
- Serializers: validate API payloads and structure API responses.
- API Views/ViewSets: expose CRUD and workflow actions through REST endpoints.
- Web Views: build template contexts and handle page-based workflows.
- Permissions: enforce action-level and object-level security rules.
- URLs: separate frontend routing from API routing.

3. Module-wise Implementation Details

- Firm and User Management: custom User model with firm association, role selection, senior-lawyer approval flag, and administrative user management.

- Client Management: supports client profile details, estate status tracking, reopen metadata, search/filter support, and client workspace pages.
- Client Assignment: links lawyers to clients and records which admin assigned them.
- Beneficiary Management: stores beneficiary identity and relationship details per client.
- Asset Management: records category, value, active status, and client linkage for estate assets.
- Asset Distribution Workflow: stores ownership allocation percentages and approval states, including approval and rejection operations.
- Document Management: links uploaded files to a client and/or asset and now serves them through a protected view.
- Audit Log: records create, update, delete, upload, approve, reject, and reopen operations with timestamps and actor metadata.

4. Query Builder / Data Access Logic

The project uses Django ORM query construction rather than raw SQL. Filtering logic is built dynamically in both API and web views based on request parameters such as search strings, status values, assigned lawyer IDs, approval state, client IDs, and asset IDs. Examples include filtering clients by search text or assigned lawyer, filtering documents by linked client or asset, filtering approvals by status, and restricting all non-superuser queries to the current firm. This ORM-driven query builder approach improves maintainability, readability, and safety.

5. Django Documentation and Framework Usage

The project makes strong use of Django's documented patterns: class-based views, `ModelViewSet`, built-in authentication, model validation through `full_clean`, Django forms, custom user models, URL routing, static/media configuration, and security middleware. It also integrates Django REST Framework and `drf-spectacular` for API schemas and documentation. The overall implementation follows documented framework conventions rather than custom infrastructure.

6. Database Design and Schema

Figure: Entity Relationship diagram of the Assetra data model.

Assetra ER Diagram

Core entities and major relationships

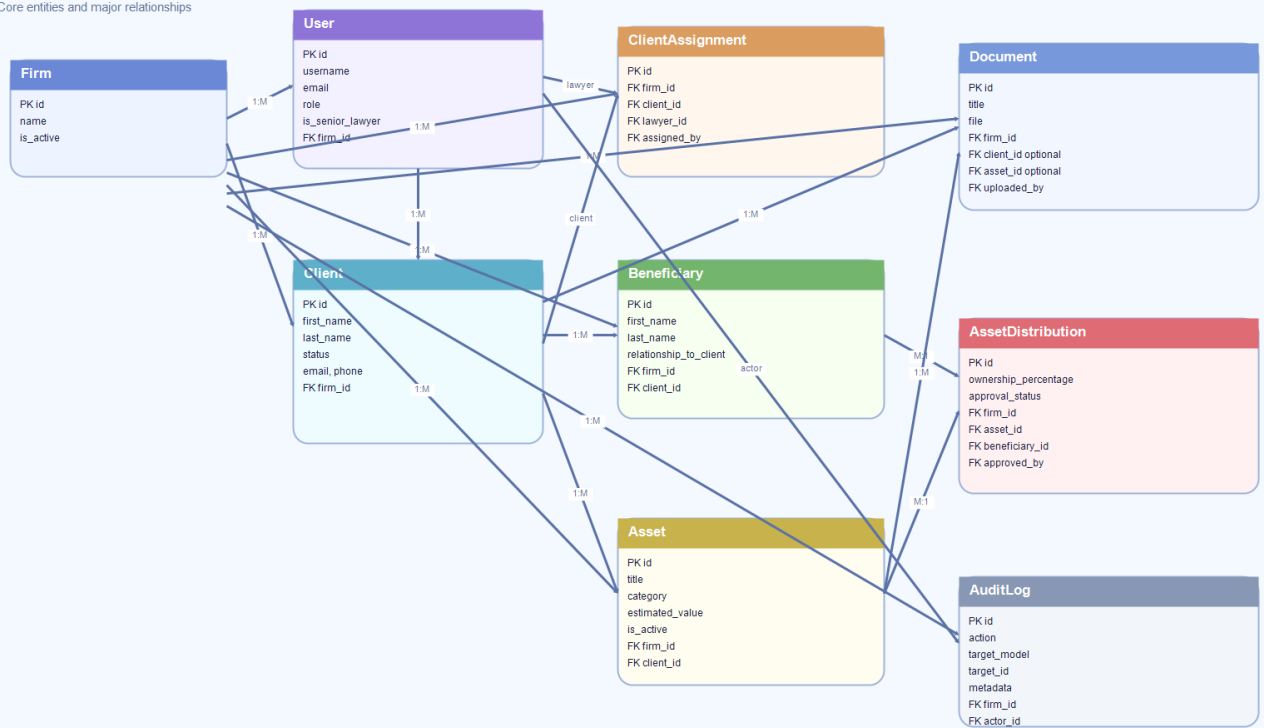
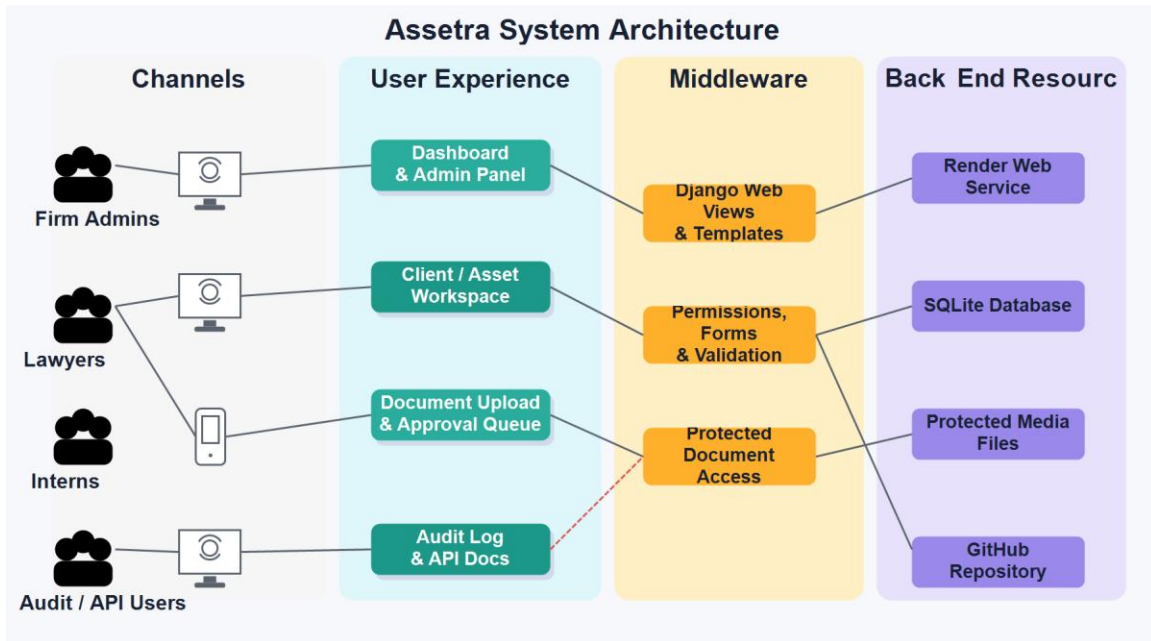


Figure: High-level system architecture of Assetra.



System Architecture Diagram:

ER Diagram:

The database is currently SQLite in local development. The schema is relational and centered on the law firm boundary. Each operational record belongs to a firm either directly or through a firm-linked parent entity.

Entity	Purpose	Key Fields	Key Relationships
Firm	Top-level tenant boundary	name, is_active	One-to-many with users, clients, assets, beneficiaries, documents, audit logs
User	Internal system user	username, role, is_senior_lawyer	Belongs to firm; may approve distributions
Client	Estate/client record	name fields, status, contact fields, notes	Belongs to firm; has assets, beneficiaries, documents, assignments
ClientAssignment	Lawyer-to-client mapping	client, lawyer, assigned_by	Belongs to firm; links client and lawyer
Beneficiary	Recipient of asset distributions	identity fields, relationship_to_client	Belongs to firm and client
Asset	Estate asset record	title, category, estimated_value, is_active	Belongs to firm and client

AssetDistribution	Ownership allocation record	asset, beneficiary, percentage, approval_status	Belongs to firm; links asset and beneficiary
Document	Uploaded supporting file	title, file	Belongs to firm; linked to client and/or asset
AuditLog	Internal traceability record	action, target_model, target_id, metadata	Belongs to firm; references actor

7. System Architecture and Design

Assetra follows a layered monolithic web architecture. The presentation layer is implemented with Django templates and class-based web views. The API layer is built with Django REST Framework viewsets and serializers. The domain layer is represented by Django models, which also contain important business rules and validation. The persistence layer uses SQLite in development through Django ORM. Cross-cutting concerns such as permissions, password hashing, audit logging, and file access control are handled through dedicated framework components.

High-level request flow: User Request -> URL Router -> View / ViewSet -> Form or Serializer Validation -> Model Rules -> Database -> Template or JSON Response.

8. Security Considerations

- Authentication: built on Django authentication with login-required access for protected areas.
- Password Security: Django password hashing is configured with Scrypt as the primary hasher and password validation rules including minimum length.
- Authorization: role-based and action-based permission checks exist through ActionRolePermission and web-view mixins.
- Firm Isolation: most queries are filtered by firm for non-superusers to avoid cross-tenant data leakage.
- Closed Estate Protection: non-admin write operations are blocked when a client/estate is closed.
- Audit Logging: sensitive create, update, delete, upload, approve, reject, and reopen events are recorded.
- Protected Documents: uploaded files are stored with randomized filenames and served through a permission-checked Django view rather than a direct file URL.

Module 4 - Integration and Deployment Documentation

1. Integration Summary

The project integrates frontend templates, web views, DRF API viewsets, serializers, models, and authentication into a unified Django project. The same domain model supports both the server-

rendered web interface and the REST API, which helps keep business rules centralized. API documentation is exposed using drf-spectacular at `/api/schema/` and `/api/docs/`. The system has also been deployed on Render for hosted access beyond the local development environment.

2. Current Deployment Details

- Hosting Platform: Render.
- Application Type: Django web application with both server-rendered pages and REST API endpoints.
- Runtime Environment: Python environment provisioned by Render during build and deploy stages.
- Web Service Start Command: typically configured using Gunicorn or an equivalent production WSGI server for Django.
- Database: the development project uses SQLite locally; production deployment on Render should use a managed production database for reliability and persistence.
- Static Files: collected and served using Django static file configuration suitable for deployment.
- Media/File Handling: uploaded documents are protected through an application route instead of direct public file URLs.
- Environment Variables: production-sensitive values such as `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS`, and database credentials should be managed through Render environment settings.
- Deployment Workflow: code is pushed to GitHub, Render pulls from the repository, installs dependencies from `requirements.txt`, applies build steps, and starts the web service.
- Operational Benefit: Render provides a simple cloud deployment path for hosting the Django application without requiring manual server provisioning.

3. How to Run the Project

GitHub repository: <https://github.com/krishAI04/assetra.git>

- Clone the repository from GitHub: <https://github.com/krishAI04/assetra.git>
- Create and activate a Python virtual environment.
- Install dependencies from `requirements.txt`.
- Run `python manage.py migrate`.
- Optionally create a superuser using `python manage.py createsuperuser`.
- Run `python manage.py runserver`.
- Open the web application at `http://127.0.0.1:8000/`.

4. Testing Strategy and Results

The current project has minimal formal automated tests in `core/tests.py`. Therefore, the testing strategy at this stage is primarily based on framework validation, runtime checks, scenario-based manual verification, and post-deployment smoke testing on Render.

- Django system validation using `python manage.py check`.
- Model-level validation through `full_clean` in forms and serializers.
- Manual validation of CRUD flows for clients, assets, beneficiaries, users, and documents.
- Manual verification of approval and rejection workflow behavior.
- Manual verification of protected document access through the new document-open endpoint.
- Template parsing and UI verification during development iterations.

Observed result summary: Django system checks passed during development changes. Major modules are structurally integrated and functional in local development. However, a stronger automated testing suite is still needed for regression safety and production readiness.

5. Limitations

- Automated test coverage is currently very limited.
- SQLite is suitable for local development but not ideal for larger production workloads.
- No asynchronous task queue or background job system is present.
- No client-facing portal exists; the system is staff-facing only.
- No document malware scanning, file content validation, or external storage integration is implemented.
- Production deployment hardening such as environment-based secret management and reverse-proxy configuration is not fully represented in the current codebase.

6. Future Enhancements

- Introduce comprehensive unit tests, integration tests, and API tests.
- Move production deployment to PostgreSQL and a hardened application server stack.
- Add email notifications for approvals, assignments, and document uploads.
- Implement richer reporting and analytics dashboards.
- Add client-facing document access or portal features where appropriate.
- Introduce advanced document validation, malware scanning, and external secure storage.
- Expand approval workflow rules and delegation support.

7. Conclusion

Assetra successfully demonstrates a structured estate and asset management platform for law firms using Django and Django REST Framework. The application covers the essential business flows of client handling, asset registration, beneficiary tracking, distribution approval, document storage, user management, and auditing within a firm-scoped architecture. The codebase already reflects important security and workflow ideas such as role-based permissions, password hashing, audit trails, and protected document access. The project has also been deployed on Render, showing that the application can be hosted beyond local development. With stronger automated testing and additional production hardening, the system can evolve into a more robust professional legal operations platform.

8. References

- Django Documentation: <https://docs.djangoproject.com/>
- Django REST Framework Documentation: <https://www.django-rest-framework.org/>
- drf-spectacular Documentation: <https://drf-spectacular.readthedocs.io/>
- Python Documentation: <https://docs.python.org/3/>
- Project source files: config/, core/, templates/, static/, and project-level configuration used to prepare this report.

Appendix - Important Project Files

File	Purpose
config/settings.py	Project configuration, installed apps, password hashers, database, static/media settings.
config/urls.py	Root routes, API docs routes, and frontend/API inclusion.
core/models.py	Database schema and core business rules.
core/forms.py	Server-rendered input validation and save logic.
core/serializers.py	API validation and response shaping.
core/views.py	REST API endpoints and workflow logic.
core/web_views.py	Server-rendered page logic.
core/permissions.py	Role-based and object-based authorization rules.
core/frontend_urls.py	Template page routes and protected document open route.
templates/	User interface structure.
static/css/assetra.css	Shared visual styling.